



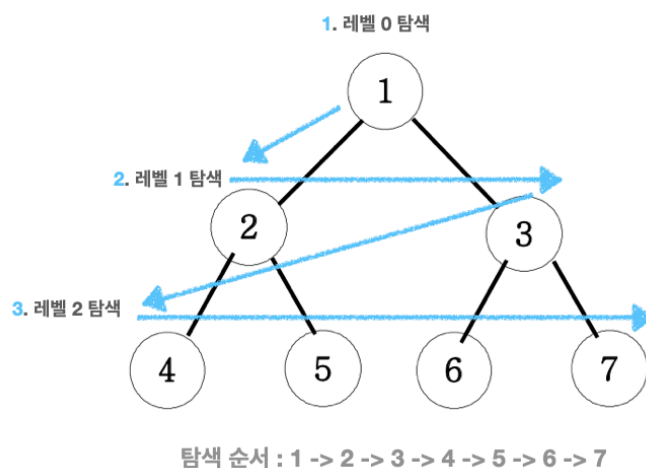
# LV17 트리(Tree) 그래프(Graph) BFS 순회

<https://youtu.be/pAu9GbtLkxY?feature=shared>

<https://youtu.be/aWZlpU14rvo?feature=shared>

<https://youtu.be/Pzhy9D9kEKA?feature=shared>

## Tree BFS 탐색 순서



## 기본 개념

- BFS는 레벨 단위로 탐색을 한다

- 한 번 만에 갈 수 있는 길을 모두 탐색한 후, 두 번 만에 갈 수 있는 모든 길을 탐색하는 방식이다

탐색 순서: 1 → 2 → 3 → 4 → 5 → 6 → 7

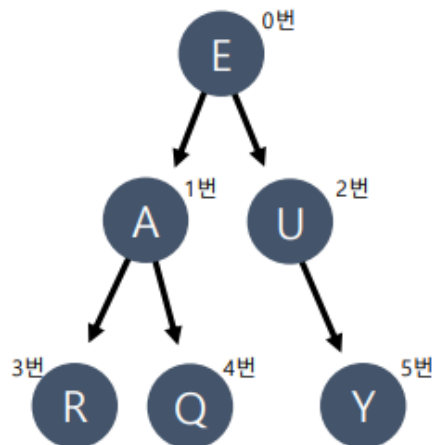
## BFS의 특징

- 두 노드사이의 최단경로를 탐색할때 활용하기 좋은 방식이다. 멀리떨어진 노드는 나중에 탐색하는 방식이기 때문!
- 큐를 활용해서 탐색할 노드의 순서를 저장하고 큐에 저장된 순서대로 탐색한다. 선입선출의 방식을 활용해야 하기 때문에 큐를 활용한다.

## BFS 구현 알고리즘

1. 루트노드에서 시작한다.
2. 루트노드와 인접하고 방문되지 않은 노드, 큐에 저장되지 않은 노드들을 Queue에 넣는다.
3. Queue에서 dequeue(pop)하여 가장 먼저 큐에 저장한 노드를 방문한다.

## 배열을 큐처럼 활용한 트리 BFS 순회 방법



## 배열을 큐처럼 활용한 트리 BFS 순회 방법

### 핵심 아이디어

- 배열을 Queue처럼 사용: **head** 와 **tail** 포인터로 큐 동작 구현
- **head**: 다음에 꺼낼 위치 (dequeue)

- **tail**: 다음에 넣을 위치 (enqueue)
- 종료 조건: `head == tail` 이 되면 큐가 비어있음

## 직접 Queue 구현

```
#include <iostream>

int map[6][6] =
{
    {0, 1, 1, 0, 0, 0},
    {0, 0, 0, 1, 1, 0},
    {0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0}
};

char value[6] = { 'E', 'A', 'U', 'R', 'Q', 'Y' };

struct Node
{
    int num;    // 노드 번호
    int level;  // 깊이 레벨
};

Node queue[7] = { {0,0} }; // 큐 역할을 할 배열, 시작 노드 {0,0} 미리 삽입
int head = 0;              // 큐에서 꺼낼 위치
int tail = 1;              // 큐에 넣을 위치 (시작노드가 이미 있으므로 1)

int main()
{
    while (head != tail)    // 큐가 비어있지 않은 동안
    {
        Node now = queue[head]; // 큐에서 노드 꺼내기 (dequeue)
        std::cout << value[now.num] << std::endl; // 현재 노드 출력
        std::cout << "-----" << std::endl;

        // 현재 노드와 연결된 모든 자식 노드들을 큐에 추가
    }
}
```

```

for (int i = 0; i < 6; i++)
{
    if (map[now.num][i] == 1) // 연결되어 있으면
    {
        queue[tail] = { i, now.level + 1 }; // 큐에 추가 (enqueue)
        tail++; // tail 포인터 이동
    }
}

head++; // head 포인터 이동 (방금 처리한 노드는 제거된 것과 같음)
}

return 0;
}

```

#### 동작 과정:

1. 시작: `queue[0] = {0,0}` , `head=0` , `tail=1`
2. E(0) 처리 → A(1), U(2)를 큐에 추가 → `tail=3`
3. A(1) 처리 → R(3), Q(4)를 큐에 추가 → `tail=5`
4. U(2) 처리 → Y(5)를 큐에 추가 → `tail=6`
5. R(3), Q(4), Y(5) 순서대로 처리

## bfs 트리 순회 `std::Queue` 라이브러리를 사용한 방법

### 표준 라이브러리의 장점

- 자동 메모리 관리: 크기 제한 없이 자동으로 확장
- 편리한 메소드: `push()` , `pop()` , `front()` , `empty()` 제공
- 안전성: 배열 경계 체크 불필요

### 표준 라이브러리 활용

```

#include <iostream>
#include <queue> // std::queue 사용을 위한 헤더

int map[6][6] =

```

```

{
    {0, 1, 1, 0, 0, 0},
    {0, 0, 0, 1, 1, 0},
    {0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0}
};

char value[6] = { 'E', 'A', 'U', 'R', 'Q', 'Y' };

struct Node
{
    int num;    // 노드 번호
    int level;  // 깊이 레벨
};

std::queue<Node> queue = {}; // 표준 라이브러리 큐 선언

int main()
{
    queue.push(Node{0, 0}); // 시작 노드를 큐에 추가

    while (!queue.empty()) // 큐가 비어있지 않은 동안
    {
        Node now = queue.front(); // 큐의 맨 앞 원소 가져오기

        std::cout << value[now.num] << std::endl; // 현재 노드 출력
        std::cout << "-----" << std::endl;

        // 현재 노드와 연결된 모든 자식 노드들을 큐에 추가
        for (int i = 0; i < 6; i++)
        {
            if (map[now.num][i] == 1) // 연결되어 있으면
            {
                queue.push(Node{ i, now.level + 1 }); // 큐에 추가
            }
        }
    }
}

```

```

        queue.pop(); // 처리 완료된 노드를 큐에서 제거
    }

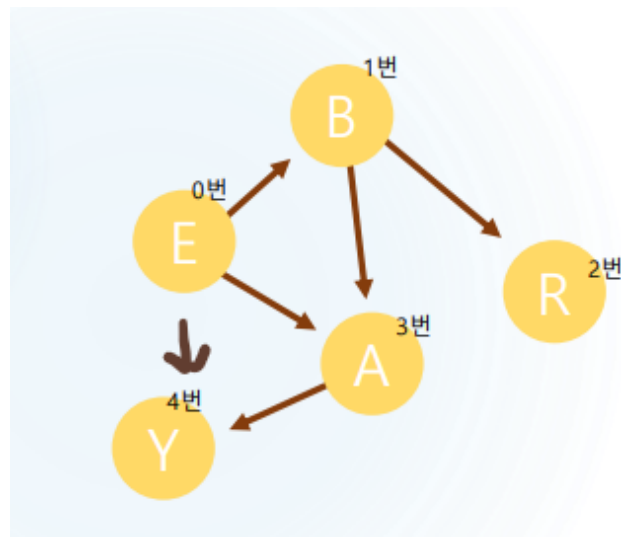
    return 0;
}

```

#### 직접 구현 vs 표준 라이브러리 비교:

- 직접 구현: `head`, `tail` 관리, 배열 크기 제한
- 표준 라이브러리: `push()`, `pop()`, `front()` 사용, 크기 제한 없음
- 성능: 거의 동일, 표준 라이브러리가 약간 더 안전

## bfs 그래프 순회 방법



#### 트리 vs 그래프의 핵심 차이점

- 트리: 사이클이 없어서 중복 방문 불가능
- 그래프: 사이클이 있어서 무한 루프 위험 → `used[]` 배열로 중복 방문 방지

#### used 배열 사용

```

#include <iostream>
#include <queue>

int map[5][5] =

```

```

{
    {0, 1, 0, 1, 1}, // E는 B, A, Y와 연결
    {0, 0, 1, 1, 0}, // B는 R, A와 연결
    {0, 0, 0, 0, 0}, // R는 연결된 곳 없음 (끝점)
    {0, 0, 0, 0, 1}, // A는 Y와 연결
    {0, 0, 0, 0, 0}, // Y는 연결된 곳 없음 (끝점)
};

char value[6] = { 'E', 'B', 'R', 'A', 'Y' };
struct Node
{
    int num; // 노드 번호
    int level; // 깊이 레벨
};

std::queue<Node> queue = {};
int used[6] = {}; // 방문 체크 배열 (0: 미방문, 1: 방문완료)

int main()
{
    queue.push(Node{0, 0}); // 시작 노드 E(0)를 큐에 추가
    used[0] = 1; // 시작 노드를 방문 처리 ★중요★

    while (!queue.empty())
    {
        Node now = queue.front(); // 현재 처리할 노드

        std::cout << value[now.num] << std::endl; // 현재 노드 출력
        std::cout << "-----" << std::endl;

        // 현재 노드와 연결된 모든 노드들 확인
        for (int i = 0; i < 5; i++)
        {
            if (used[i] == 1) // 이미 방문한 노드면 건너뛰기
                continue;
            if (map[now.num][i] == 0) // 연결되지 않은 노드면 건너뛰기
                continue;
        }
    }
}

```

```

        used[i] = 1;                // 방문 처리 ★중요★
        queue.push(Node{ i, now.level + 1 });    // 큐에 추가
    }

    queue.pop(); // 현재 노드 처리 완료
}

return 0;
}

```

### 핵심 포인트:

1. 시작할 때: `used[0] = 1` 로 시작점 방문 처리
2. 큐에 넣을 때: `used[i] = 1` 로 미리 방문 처리 (중요!)
3. 두 번의 **continue**: 방문했거나 연결되지 않은 노드 건너뛰기

### 동작 과정:

1. E(0) 처리 → B(1), A(3), Y(4) 발견하여 큐에 추가 및 방문 처리
2. B(1) 처리 → R(2) 발견하여 큐에 추가 (A는 이미 방문 처리됨)
3. A(3) 처리 → Y(4)는 이미 방문 처리되어 건너뛴
4. Y(4), R(2) 순서대로 처리하며 종료

### 왜 큐에 넣을 때 바로 방문 처리?

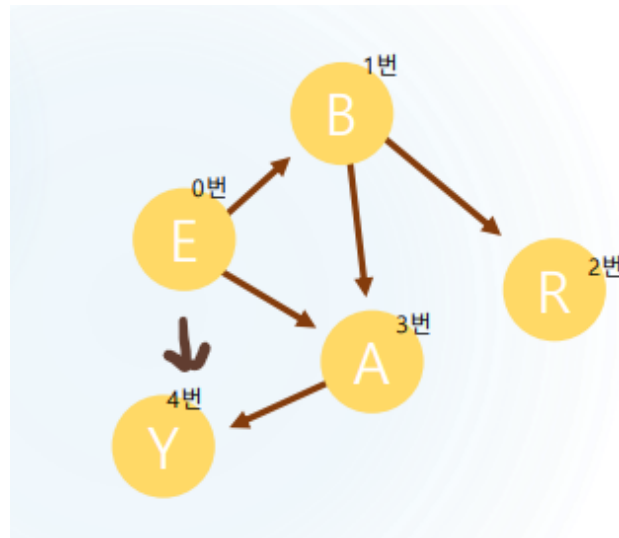
- 큐에서 꺼낼 때 방문 처리하면, 같은 노드가 여러 번 큐에 들어갈 수 있음
- 미리 방문 처리하면 중복 추가 방지 가능

## 핵심 비교

| 구분    | DFS        | BFS         |
|-------|------------|-------------|
| 탐색 방식 | 깊이 우선      | 너비 우선       |
| 자료구조  | Stack (재귀) | Queue       |
| 구현    | 재귀함수       | 반복문 + Queue |
| 메모리   | 적음         | 많음          |
| 경로 탐색 | 모든 경로      | 최단 경로       |



## 그래프 BFS 탐색순서



### 핵심 개념

그래프도 트리순회와 크게 다른 점은 없다. 다만 이미 방문한 노드를 2번 방문하지 않기 위한 예외처리만 진행해주면 된다!

### bfs 그래프 순회 방법

```
#include <iostream>
#include <queue>

int map[5][5] =
{
    {0, 1, 0, 1, 1},
    {0, 0, 1, 1, 0},
    {0, 0, 0, 0, 0},
    {0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0},
};

char value[6] = { 'E', 'B', 'R', 'A', 'Y' };
struct Node
{
    int num;
    int level;
};
```

```

std::queue<Node> queue = {};
int used[6] = {};

int main()
{
    queue.push(Node{0, 0});
    used[0] = 1;

    while (!queue.empty())
    {
        Node now = queue.front();

        std::cout << value[now.num] << std::endl;
        std::cout << "-----" << std::endl;

        for (int i = 0; i < 5; i++)
        {
            if (used[i] == 1)
                continue;
            if (map[now.num][i] == 0)
                continue;

            used[i] = 1;
            queue.push(Node{ i, now.level + 1 });
        }

        queue.pop();
    }

    return 0;
}

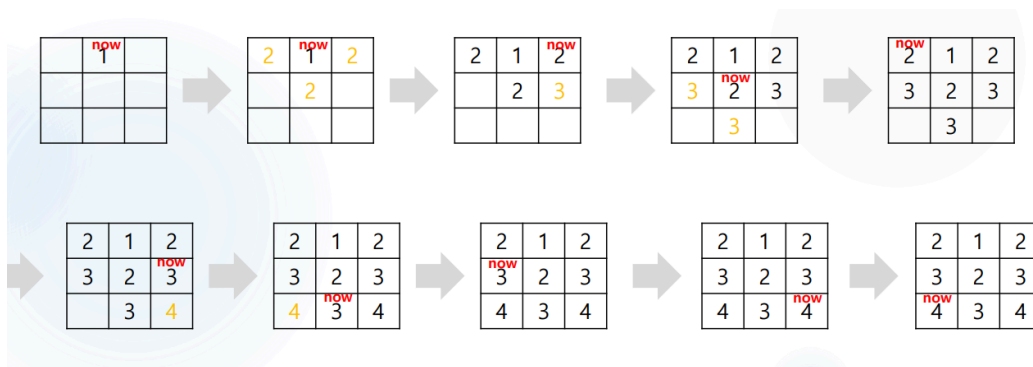
```

## BFS 길찾기 맵 탐색

### 기본 개념

**bfs** 알고리즘은 단순 순회 외에도 길찾기 알고리즘으로 활용 하는 경우도 많다.

다만 여기에서는 당장 길찾기에 활용 하면 목적할 수 있게에 2차원 배열을 bfs를 이용해 순회 하는 방법부터 연습해보도록 하자다. (다음 시간에 길찾기 bfs활용)



## BFS 2차원 배열 길찾기 사전준비 ( 2차원배열 BFS로 순회하기 )

```
#include <iostream>
#include <queue>

int map[3][3] =
{
    {0, 0, 0},
    {0, 0, 0},
    {0, 0, 0},
};

struct Node
{
    int x;
    int y;

    int level;
};

std::queue<Node> queue = {};
int used[6] = {};

int direct[4][2] =
{
    0, 1,
    0, -1,
```

```

    1, 0,
    -1, 0,
};

int main()
{
    queue.push(Node{1, 0, 1});
    map[0][1] = 1;

    while (!queue.empty())
    {
        Node now = queue.front();

        if (now.y == 2 && now.x == 2)
        {
            int t = 0;
        }

        for (size_t i = 0; i < 4; i++)
        {
            int dy = now.y + direct[i][0];
            int dx = now.x + direct[i][1];

            if (dy < 0 || dx < 0 || dy > 2 || dx > 2)
                continue;
            if (map[dy][dx] > 0)
                continue;

            Node next = Node{ dx, dy, now.level + 1 };
            map[dy][dx] = next.level;

            queue.push(next);
            int a = 0;
        }

        queue.pop();
    }
}

```

```
return 0;
}
```

## "강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

## 그래서, 얀얀코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얀얀코딩 코칭에서는 단순한 예제가 아닌,

**스스로 문제를 분석하고 구현해야 하는 연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 **실제 구현 능력**
- 기능 단위가 아닌, **프로그램 단위로 사고하는 습관**
- 마침내 **자신의 힘으로 코드를 끝까지 작성하는 경험**

## 지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: 얌얌코딩 상담방

▼ oopy:hide

## 문제풀이

### 문제 1번

문자열로 구성된 이진트리를 입력 받아주세요.

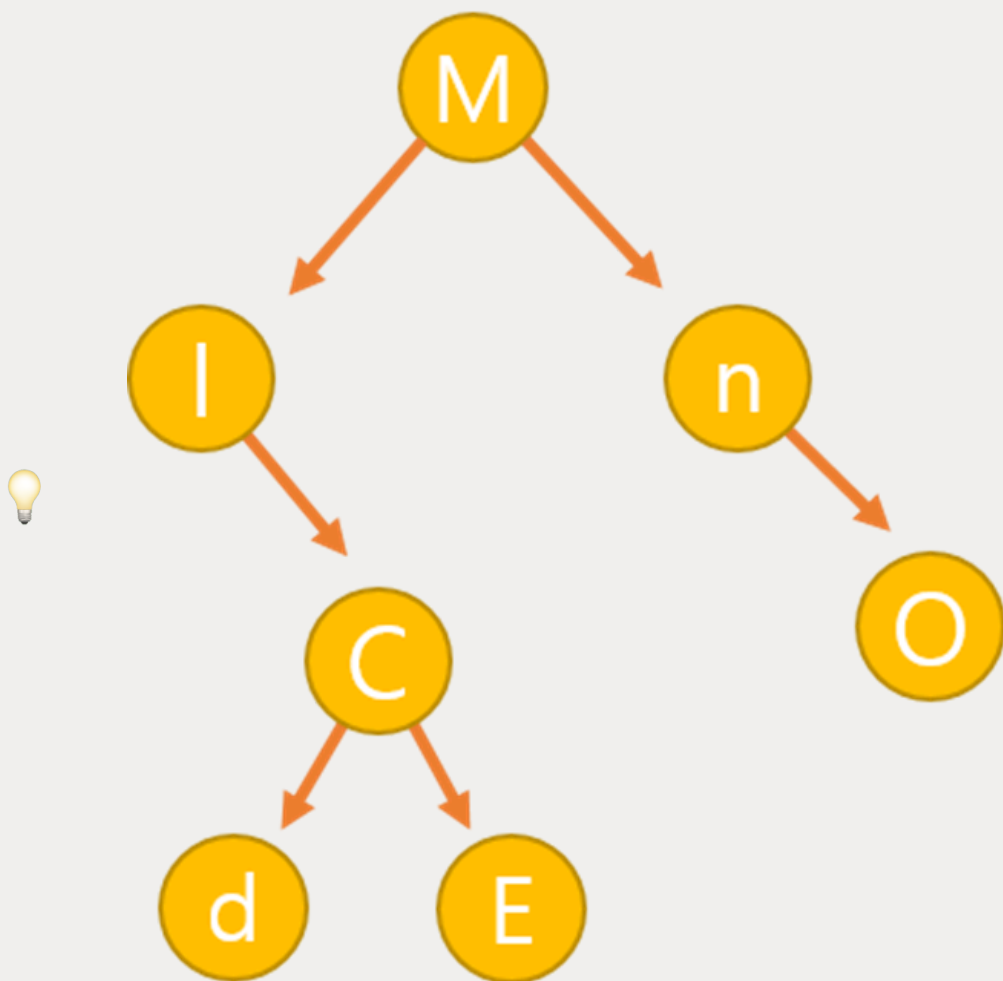
(1차원 배열에 저장하는 형태로 입력이 주어집니다.)

(#은 노드가 없음을 의미합니다.)

Root노드부터 DFS를 돌리면서

대문자 노드를 탐색할 때마다 출력 해 주세요.

만약 #MIn#C#O##dE를 입력받았다면



출력결과 : MICEO

### 입력 예제

#MIn#C#O##dE

## 출력 결과

MICEO



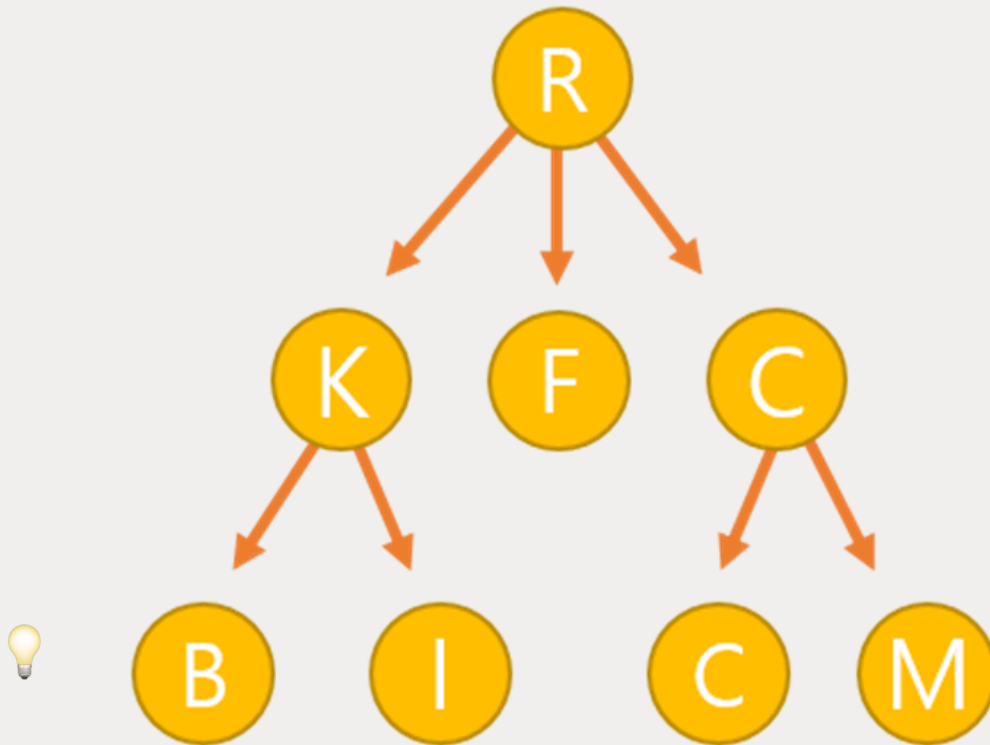
## 문제 2번

8개의 노드로 구성 된 문자열과

대응되는 인접행렬을 입력받아주세요.

아래 이미지는 입력 예시에 해당하는 트리입니다.

0번 노드부터 DFS로 노드들을 탐색하면서 출력 해 주세요.



## 입력 예제

RKFCBICM

01110000

00001100

00000000

00000011

00000000

00000000

00000000

00000000

## 출력 결과

RKBIFCCM

### 문제 3번

6글자를 입력받고 링크드리스트로 다음과 같이 연결시켜주세요.

ex)

```
head = new Node( );
```

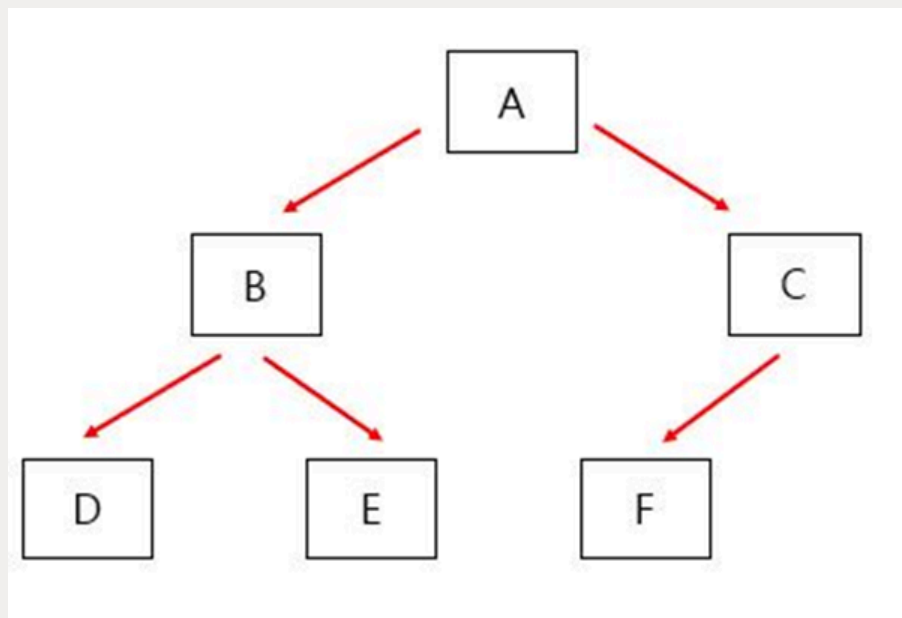
```
head→left = new Node( );
```

```
head→right = new Node( );
```

...

링크드리스트 노드에 각각 입력받은 문자를 넣어주면 됩니다.

만약, ABCDEF 를 입력 받았다면 아래와 같습니다.



이제 DFS를 돌린결과를 출력 해주세요.

### 입력 예제

ABCDEF

### 출력 결과

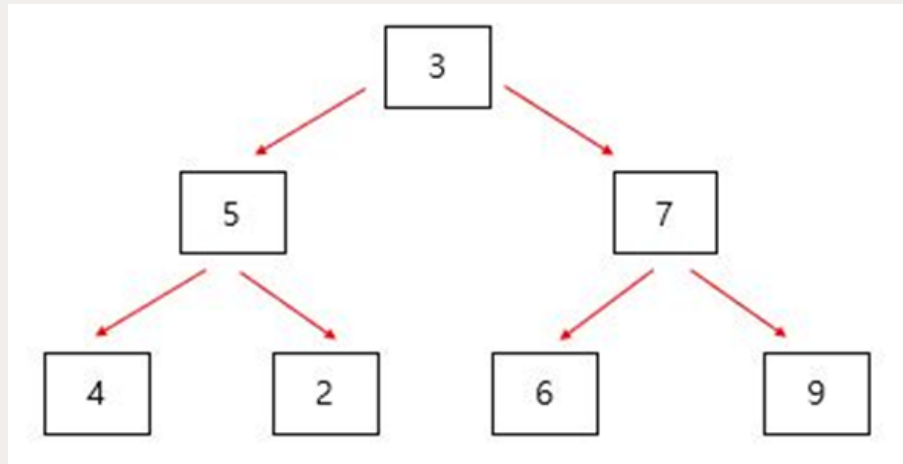
ABDECF

#### 문제 4번

노드에 들어갈 숫자 7개를 입력 받으세요.

만약 **3 5 7 4 2 6 9** 를 입력 받았다면 다음과 같습니다.

(Root는 1번 index에 저장해야 함을 잊지마세요)



이제 **마지막 노드(Level2)에 있는 숫자들의 합을 DFS를 돌려 구해주세요.**

#### 입력 예제

3 5 7 4 2 6 9

#### 출력 결과

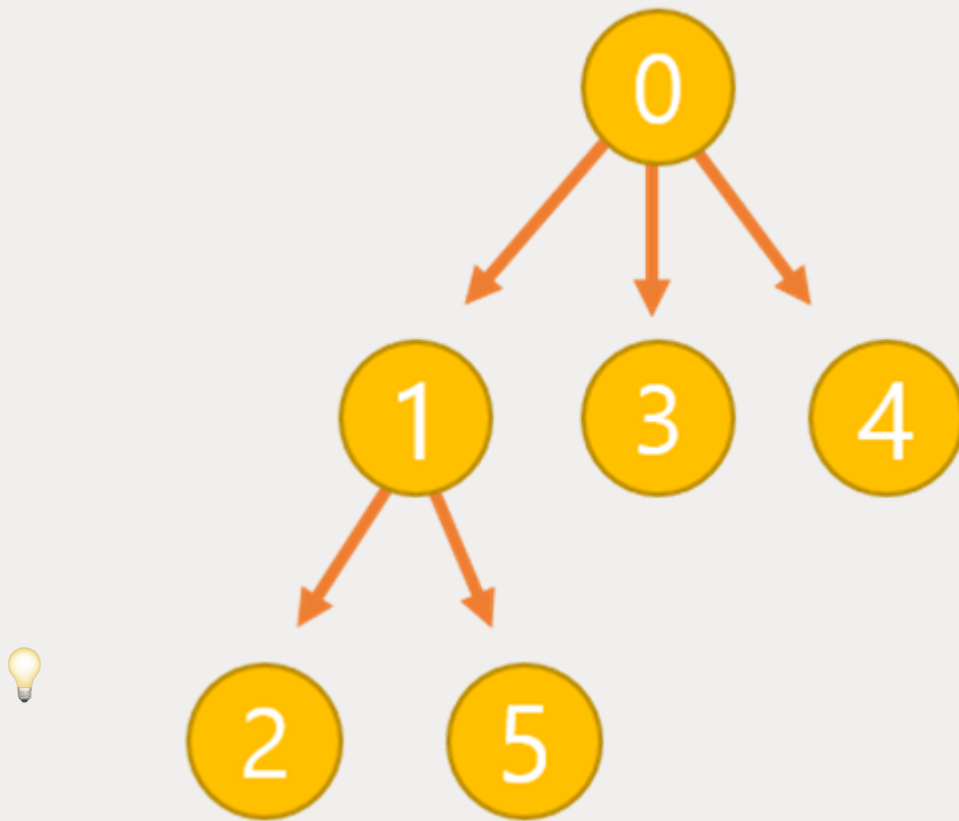
21

### 문제 5번

0 ~ 5번까지 6개 노드로 구성된 인접행렬을 입력받아주세요.

0번 노드부터 BFS를 돌려, **홀수 노드를 찾을 때 마다 출력** 해 주세요.

만약 위와 같이 트리를 입력받았다면,



**출력결과 : 1 3 5**

### 입력 예제

0 1 0 1 1 0

0 0 1 0 0 1

0 0 0 0 0 0

0 0 0 0 0 0

0 0 0 0 0 0

0 0 0 0 0 0

## 출력 결과

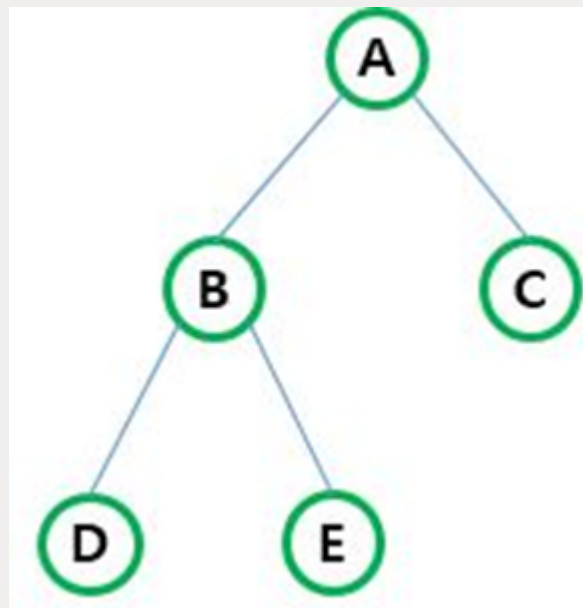
1 3 5

### 문제 6번

노드 안에 들어갈 문자 5개를 입력 받으세요.

만약 ABCDE 입력 받으면 아래와 같이 저장됩니다.

(트리모양은 고정입니다)



위 트리를 **인접행렬**로 저장하고, DFS를 돌려 탐색 순서대로 출력하세요.

(힌트 : 입력받은 문자열은 배열에 따로 저장해두어야 합니다)

## 입력 예제

ABCDE

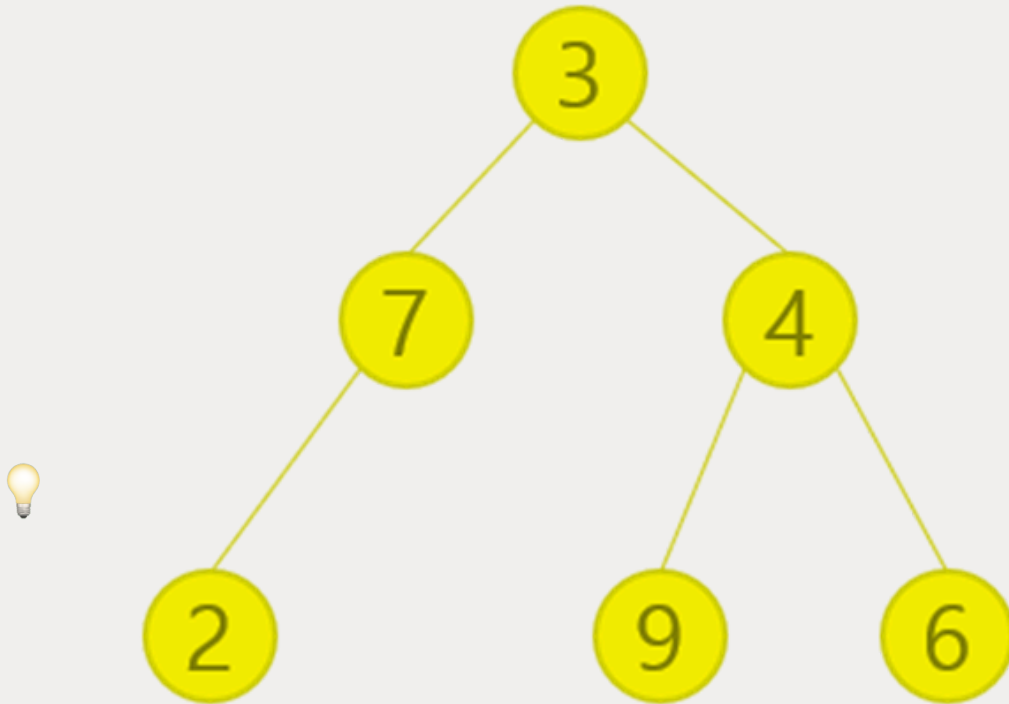
## 출력 결과

ABDEC

### 문제 7번

총 8개의 숫자를 1차원 배열에 입력받습니다. (1~9 사이 숫자, 0은 없는 노드)

BFS를 돌려 탐색 순서대로 값을 출력 해주세요.



만약 **0 3 7 4 2 0 9 6** 을 입력 받았다면 위와 같은 트리가 됩니다.

1. ex) **0 3 7 4 2 0 9 6** ⇒ 출력결과 : **3 7 4 2 9 6**

### 입력 예제

0 1 2 3 0 0 4 5

### 출력 결과

1 2 3 4 5

### 문제 8번

터미네이터의 신경체계가 다음과 같이 트리형태로 되어있다고 합니다.  
인접행렬 형태로 초기화를 해 주세요.



이 신경체계를 트리로 하드코딩 하고, 나노 탐사로봇이 탐색을 하려고 합니다.

BFS 알고리즘을 쓸때 탐사순서대로 출력해주세요.

예를들어 E를 입력받았다면,

E에서 출발하는 BFS를 돌리면 됩니다. (출력결과 : E H J)

예를들어 B를 입력받았다면,

B에서 출발하는 BFS를 돌리면 됩니다. (출력결과 : B C D E F G H I J)



## 입력 예제

A

## 출력 결과

A B C D E F G H I J

### 문제 9번

조상 두더지는 1년에 세마리의 자식을 낳습니다.

1년 후 각각의 자식들은 각자 세마리의 자식을 낳습니다.

큐를 이용해서  $n$ 년 후에는 총 몇마리의 두더지가 있는지 출력 해주세요.



ex)

0년 => 1마리

1년 => 3마리 + 1마리 = 4마리

2년 => 9마리 + 3마리 + 1마리 = 13마리



[힌트]

Queue에 1을 넣어두고 시작합니다.

Head index의 값에 \* 3 곱한 값을 큐에 추가하고, Head++ (큐에 3이 적혀 집니다)

Head index의 값에 \* 3 곱한 값을 큐에 추가하고, Head++ (큐에 9가 적혀 집니다)

이 과정을  $n$ 번 반복한 후

큐에 적혀있는 숫자들을 모두 더하면 됩니다.

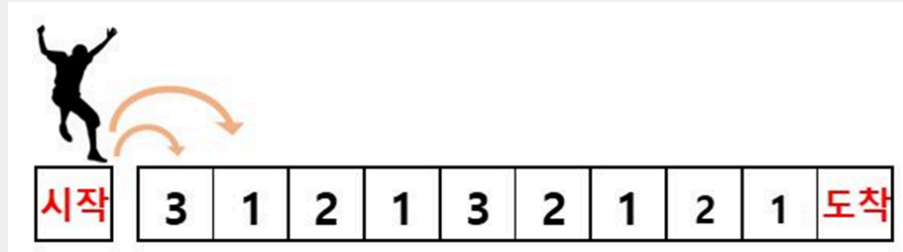
### 입력 예제

1

### 출력 결과

4

## 문제 1번



처음 점프를 할  $n$ 값을 입력 받으세요.

만약,

1을 입력 받으면 3으로 점프하고,

2를 입력 받으면 1로 점프합니다.

그리고 다음 점프는 바닥에 써 있는 칸만큼 점프를 계속 합니다.

도착지점에 도달하면 return을 하게 되어 시작점으로 돌아옵니다.



이 과정을 모두 출력 해주세요.

(재귀호출로 구현 해 주세요)

ex)

<입력>

1

<출력>

시작 3 1 3 2 도착 2 3 1 3 시작

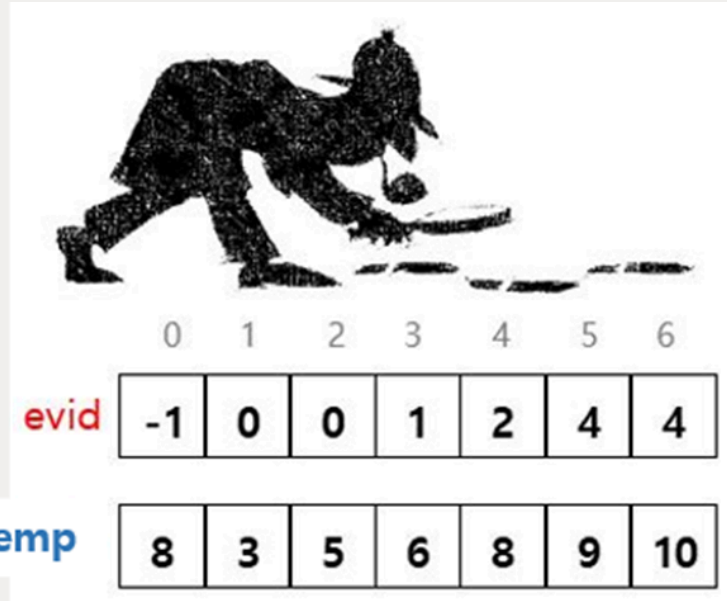
## 입력 예제

5

## 출력 결과

시작 3 2 도착 2 3 시작

## 문제 2번



추적을 시작 할 index를 입력 받으세요.

만약 5를 입력 받았다면, **5번 index** 부터 추적을 시작합니다.

5번 index를 살펴보면 범인은 4번 index에서 출발했고, 9시에 도착했다는 것을 알 수 있습니다.



4번 index를 살펴보면 범인은 2번 index에서 출발했고, 8시에 도착했다는 것을 알 수 있습니다.

2번 index를 살펴보면 범인은 0번 index에서 출발했고, 5시에 도착했다는 것을 알 수 있습니다.

범죄자의 흔적들을 추적해가면, 마지막에는 -1에 도달합니다.

- 1이 있는 곳에서 범죄자를 잡을 수 있습니다.

**범인은 0번 index부터 몇 시에 몇 번 index로 이동했는지**

**순서대로 출력하세요.**

(재귀를 이용해서 범인을 추적 해 주세요)

## 입력 예제

5

## 출력 결과

0번index(출발)

2번index(5시)

4번index(8시)

5번index(9시)

### 문제 3번

3×3 배열에 숫자를 입력해 채워줍니다.

그리고 가로로 한줄씩 모두 같은 숫자인지 검사하는 프로그램을 작성해주세요.

같으면 같은 숫자를 출력, 아니면 (소문자)x를 출력 하세요.



|   |   |   |
|---|---|---|
| 3 | 3 | 3 |
| 5 | 6 | 7 |
| 9 | 9 | 9 |

ex)

<입력>

<출력>

3 3 3

3

5 6 7

x

9 9 9

9

### 입력 예제

3 3 3

5 6 7

9 9 9

### 출력 결과

3

x

9

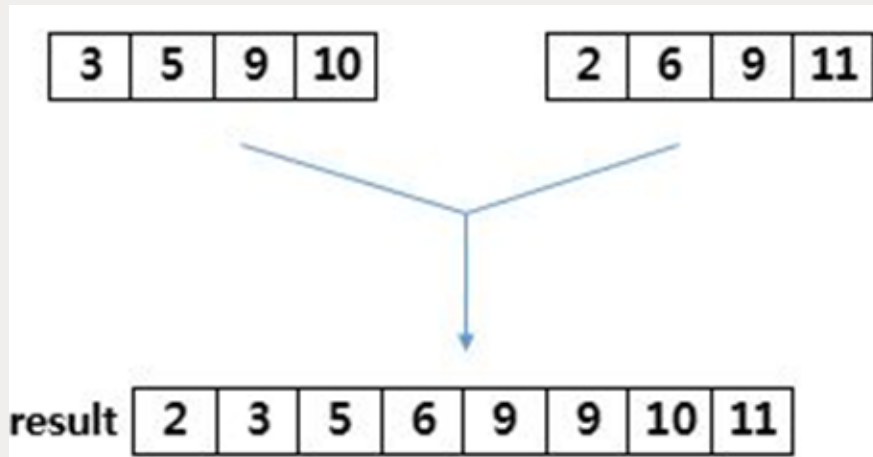
#### 문제 4번

숫자 4개씩 2개의 배열에 숫자를 입력 받아주세요.



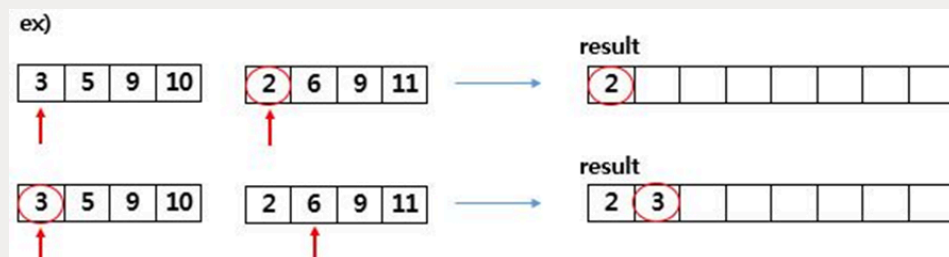
입력값은 정렬된 상태로 숫자가 들어옵니다.

이 두배열을 합쳐 정렬된 8개의 숫자를 저장 하려고 합니다.



이렇게 합치기 위한 알고리즘은

비교를 한 후에 작은 숫자를 result배열에 넣고 화살표를 옆으로 옮깁니다.



위와 같은 동작을 반복하면, 정렬된 result 배열을 만들 수 있습니다.

위 알고리즘대로 코딩하여 result 배열을 만들고 출력 해주세요.

#### 입력 예제

3 5 9 10

2 6 9 11

## 출력 결과

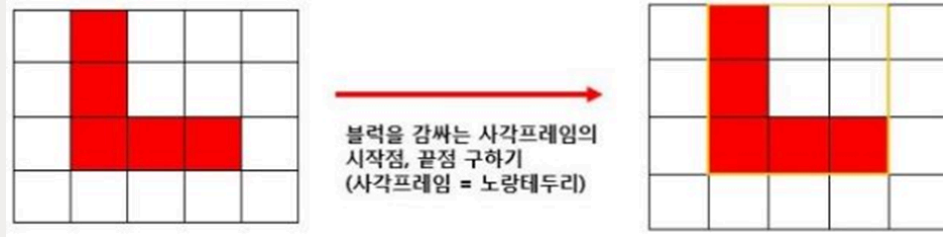
2 3 5 6 9 9 10 11

### 문제 5번

4×5 2차배열이 있습니다.

블럭을 입력 받아주세요.

사각프레임에 블럭이 있을때, 이블럭을 감싸는 **사각프레임의 시작점과 끝점**의 좌표를 구해주세요.



ex1)

[입력]

0 1 0 0 0

0 1 0 0 0

0 1 1 1 0



0 0 0 0 0

ex2)

[입력]

0 0 0 0 0

0 0 1 1 0

0 0 1 1 1

0 0 0 0 0

[출력]

(0,1)

(2,3)

[출력]

(1,2)

(2,4)

### 입력 예제

0 1 0 0 0

0 1 0 0 0

0 1 1 1 0

0 0 0 0 0



## 출력 결과

(0,1)

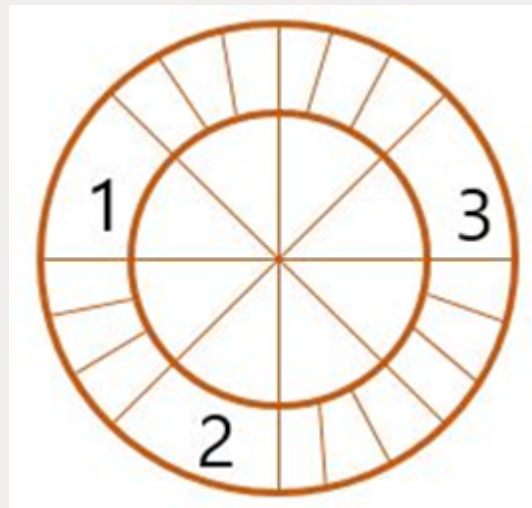
(2,3)

### 문제 6번

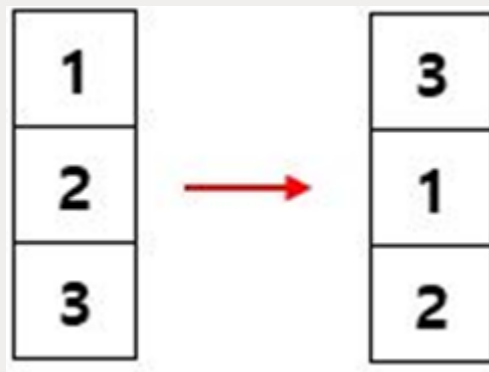
3개의 숫자로 되어있는 톱니바퀴가 있습니다.



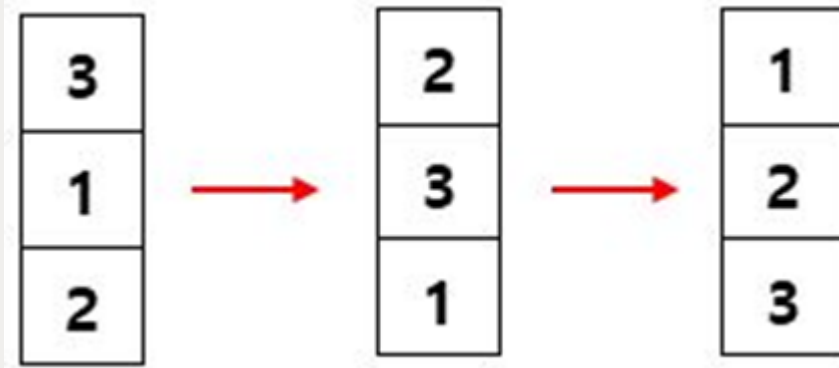
그림으로 그리면 아래와 같습니다.



이 상태에서 아래쪽으로 한칸 돌리면 아래와 같이 됩니다.



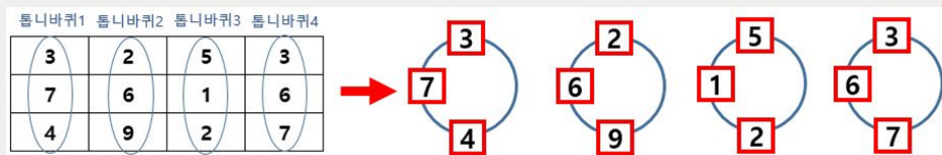
이 상태에서 두번 더 돌리면 아래와 같이 됩니다.



이런식으로 동작하는 톱니바퀴가 4개 있습니다.

각 톱니바퀴를 돌려 결과를 출력 하려고 합니다.

아래의 톱니바퀴 상태 배열을 하드코딩 해 주세요.



그리고 몇번 돌릴지에 대한 숫자 4개를 입력 받아주세요.

이 숫자 4개는 4개의 톱니바퀴를 각각 돌릴 횟수입니다.

숫자대로 톱니바퀴를 돌렸을 때 나온 결과를 출력 해주세요.

## 입력 예제

1 2 1 2

## 출력 결과

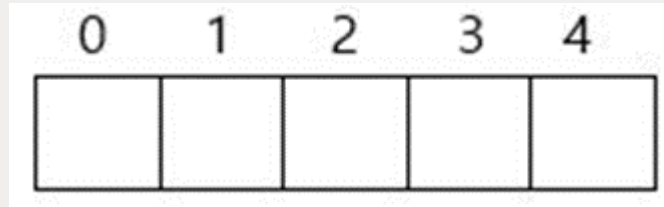
4626

3957

7213

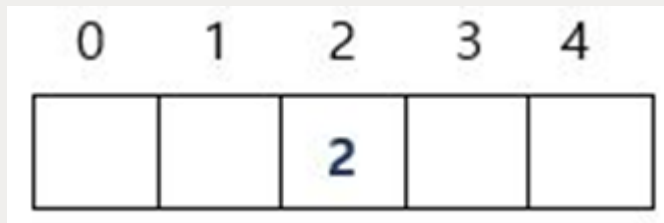
### 문제 7번

다섯칸의 맵이 있고 지렁이를 올려두려고 합니다.

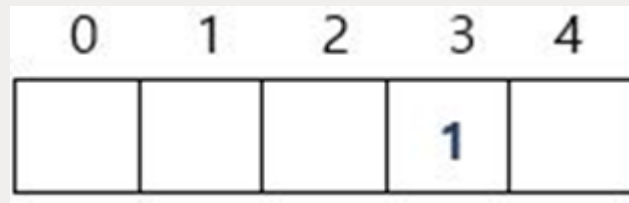


수명이 2인 지렁이를 2번 index에 올려두면 아래와 같이 됩니다.

배열안에 적은 값이 지렁이의 수명입니다



지렁이는 1초에 한번씩 오른쪽으로 한칸씩 이동합니다.

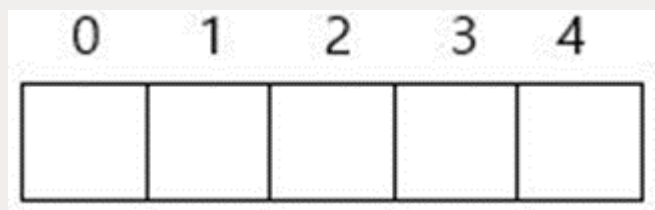


한칸씩 이동하면서 수명이 1씩 줄어 듭니다.

다음 1초 후에는 수명이 0이되어 지렁이는 죽게 됩니다.

만약 지렁이가 수명이 남았지만 맵 밖으로 나가도 죽게 됩니다.

(죽은 후에는 숫자 0을 표기하지 않습니다)



올려놓을 지렁이의 index와 수명을 입력 받고  
지렁이가 죽을 때 까지 동작 결과를 출력하세요.

## 입력 예제

2 2

## 출력 결과

\_2\_

\_\_1\_

\_\_\_\_

### 문제 8번

아래 배열에 게임 상태를 나타내는 MAP입니다.

색칠된 부분은 벽이고 **알파벳은 몬스터의 이름**입니다.

몬스터의 AI는 단순하여



만 순서대로 반복해서 움직입니다.

1초 후에는 오른쪽으로 움직이고,

2초 후에는 아래로 움직이고,

3초 후에는 왼쪽,

4초 후에는 위,

5초 후에는 다시 오른쪽으로 움직입니다.

이 몬스터는 벽을 통과하거나 몬스터끼리 겹치지 못합니다.

그래서 움직이려고 하는 곳이 막혀있다면 가만히 서있습니다.

- 몬스터는 알파벳 순서대로 움직이게 됩니다.



아래 예제에서는 A먼저 움직이고, C움직이고, D가 움직이게 됩니다.

MAP을 입력 받고, 5초 후 상황을 출력 해주세요.

- 모든 몬스터는 1초에 한번씩 움직이게 됩니다.



### 입력 예제

\_A\_

#\_D

C\_#

#\_

## 출력 결과

\_A

#\_D

\_C#

#\_